PYTHON BACKEND ENGINEERING

Getting Started with Python & Git

Class 01 · Complete Student Notes

LEVEL
Beginner → Curious

TODAY'S TOOLS
Python + Git

INSIDE
Concepts · Examples · Practice

> **How to use these notes.** Everything is here — concepts, worked examples, the errors you'll hit and how to fix them, and “Go Deeper” boxes that reveal how Python actually works under the hood. Read actively: type every example yourself, break things on purpose, and solve the green practice boxes.

1 What Are We Learning?

Backend engineering is the “behind-the-scenes” part of any app. When you open an app and see your posts, or order food, a program on a server is fetching your data, processing it, saving it, and sending it back. That server-side logic is the **backend** — and that's what this course teaches you to build.

Here is the full road ahead. We're at the very first stop:



We start with the **Python** language, plus **Git** — the tool every professional uses to save and manage code. Why Git from day one? Because building the habit of versioning your work early is what separates hobby code from engineering.

2 Setting Up Your Machine

2.1 Check / Install Python

Open your terminal (Command Prompt on Windows, Terminal on Mac/Linux) and type:

```
TERMINAL
python --version
```

If installed you'll see `Python 3.12.0`. Otherwise download it from **python.org**, and on Windows tick **“Add Python to PATH”** during install.

> **Note**
> On some systems the command is `python3`, not `python`. If one fails, try the other.

2.2 Install a Code Editor (VS Code)

We recommend **Visual Studio Code** — free and industry-standard. After installing, add the **Python extension by Microsoft** for autocomplete, error highlighting, and a Run button.

2.3 Terminal Basics

Command	What it does
`pwd`	Print the folder you're currently in
`ls` (dir on Windows)	List files and folders here
`cd myfolder`	Move into a folder
`cd ..`	Move back up one folder
`mkdir myfolder`	Create a new folder

2.4 Two Ways to Run Python

The REPL — an interactive shell where each line runs instantly. Great for quick tests:

PYTHON REPL

```
python
>>> 2 + 3
5
>>> exit()
```

Script files — real programs live in `.py` files. Write, save, then run the whole file:

TERMINAL

```
python hello.py
```

3 Your First Program

Create `hello.py`:

```
HELLO.PY
print("Hello, World!")
```

Run `python hello.py` → it prints `Hello, World!`. Print multiple things with commas:

```
PYTHON
print("The answer is", 42)
# Output: The answer is 42
```

PRACTICE 3

1. Print your name and your coding goal on two separate lines.

4 Comments

Anything after `#` is ignored by Python — notes for humans.

```
PYTHON
# this whole line is ignored
print("Hello")   # the note after # is ignored, the print runs
```

Note

Good comments explain *why*, not *what* is obvious from the code.

5 Variables

A variable is a name that points to a value. Create one with `=`:

```python
name = "Rahim"
age = 25
print(name, age)   # Rahim 25
```

You can reassign any time. `=` means “point this name at that value”:

```python
score = 10
score = score + 5
print(score)   # 15
```

Naming Rules

- Letters, numbers, underscores only — **cannot start with a number**.
- No spaces → `first_name`, not `first name`.
- Case-sensitive → `age` ≠ `Age`.
- Use meaningful names → `total_price` beats `x`.

> **Convention**
> Python uses **snake_case** — lowercase with underscores, like `user_age`.

> 🔬 **Go Deeper — a variable is a label, not a box**
> In many languages a variable is a box that holds a value. In Python it's a **label pointing at an object** in memory. So `a = b` makes both names point to the *same* object — no copy is made. We'll see why this matters in Section 11.

> **PRACTICE 5**
> 1. Create `name`, `price`, `quantity` and print them.
> 2. Make `total = price * quantity` and print it.

6 Data Types

Type	Meaning	Example
`int`	Whole number	`25, -7, 1000`
`float`	Decimal number	`99.50, 3.14`
`str`	Text (in quotes)	`"Rahim"`
`bool`	True or False	`True, False`

Check any value's type with `type()`:

PYTHON

```python
print(type(25))       # <class 'int'>
print(type("Rahim"))  # <class 'str'>
```

Note

`"25"` is text; `25` is a number. You can do math with `25` but not with `"25"`.

🔬 **Go Deeper — why does 10 / 3 give a float?**

In Python 3, the `/` operator *always* produces a float, even `10 / 2` gives `5.0`, not `5`. If you want a whole number, use floor division `//`. This surprises many beginners.

7 Indentation — Python's Big Idea

Most languages group code with curly braces `{ }`. Python is different: it groups code by **indentation** (the spaces at the start of a line). The indentation isn't decoration — it *is* the syntax. Lines indented the same amount belong to the same block.

```python
age = 20
if age >= 18:
    print("You are an adult")   # indented → inside the if
    print("Welcome!")           # still inside the if
print("This always runs")       # not indented → outside the if
```

The standard is **4 spaces** per level. Be consistent — don't mix tabs and spaces. (VS Code inserts 4 spaces when you press Tab, so you're safe.)

IndentationError

Forgetting to indent after a colon is one of the first errors you'll meet:

```python
if age >= 18:
print("hi")   # not indented!

# IndentationError: expected an indented block after 'if'
```

Fix: indent the line under the `if` by 4 spaces.

🔬 **Go Deeper — why did Python choose this?**

The creator, Guido van Rossum, argued that programmers indent their code for readability anyway — so why not make that indentation meaningful and drop the braces entirely? The result is code that's forced to stay visually clean. Two developers writing the same Python tend to produce very similar-looking files.

8 Operators

8.1 Arithmetic

```python
print(10 + 3)    # 13
print(10 / 3)    # 3.333...  division → float
print(10 // 3)   # 3    floor division (drops decimal)
print(10 % 3)    # 1    modulo (remainder)
print(10 ** 2)   # 100  exponent
```

Handy trick

`n % 2 == 0` is the standard way to check if a number is even.

8.2 Comparison (returns True/False)

```python
print(5 > 3)    # True
print(5 == 5)   # True   (equal?)
print(5 != 3)   # True   (NOT equal?)
```

Most common beginner bug

`=` *assigns*; `==` *compares*. `x = 5` stores 5; `x == 5` asks whether x equals 5.

PRACTICE 8

1. Print `17 % 5` and `17 // 5` and explain each result to yourself.
2. Check whether a number is even using `%`.

9 Getting Input From the User

`input()` waits for the user to type. Whatever they type comes back as a **string** — always.

```python
name = input("What is your name? ")
print("Hello,", name)
```

Need a number? Convert with `int()` (or `float()` for decimals):

```python
age = int(input("Enter your age: "))
print("Next year you'll be", age + 1)
```

Watch out

Without `int()`, `age` is text and `age + 1` crashes. See exactly why in the next section.

10 Errors & How to Fix Them

Errors are normal — every engineer hits dozens a day. The skill isn't avoiding them, it's **reading the message**. Python always tells you the error *type* and the *line*. Let's meet the five you'll see most, and fix each one.

A worked example — the classic TypeError

PYTHON — BUGGY

```python
age = input("Age: ")   # input is always a string, e.g. "25"
print(age + 1)          # trying to add a number to text

# TypeError: can only concatenate str (not "int") to str
```

Why: `age` holds the text `"25"`, and Python won't add a number to text. **Fix:** convert the input to a number first.

PYTHON — FIXED

```python
age = int(input("Age: "))   # now age is the number 25
print(age + 1)               # 26 ✓
```

The five errors you'll meet first

Error	Cause	Fix
`SyntaxError`	Typo — missing colon, quote, or bracket	Re-read the flagged line carefully
`NameError`	Using a variable that doesn't exist (often a typo)	Check the spelling / define it first
`TypeError`	Operation on the wrong type (str + int)	Convert with `int()` / `str()`
`ValueError`	Right type, impossible value (`int("hi")`)	Feed it something convertible
`IndentationError`	Wrong or missing indentation	Align blocks to 4 spaces

See each one crash

```python
print("hi"          # SyntaxError: missing closing )

print(nam)           # NameError: name 'nam' is not defined
                     # (the variable is 'name', not 'nam')

int("hello")         # ValueError: "hello" isn't a number
```

Read errors bottom-up

Python prints a “traceback”. The **last line** names the error type and message — start there. The line above shows exactly where it happened.

🔬 **Go Deeper — errors are objects too**

Every error is actually a Python object (an *exception*). Later you'll learn to *catch* them with `try / except` so your program can recover instead of crashing — for example, asking again when the user types letters instead of a number.

PRACTICE 10

1. On purpose, write code that triggers a `TypeError`, read the message, then fix it.
2. Do the same for a `NameError`.

11 How Python Manages Memory

You never manually reserve or free memory in Python — it's automatic. But understanding *how* makes many “weird” behaviours suddenly obvious. Here's the mental model.

11.1 Names point to objects

When you write `a = 5`, Python creates an object `5` in memory and points the name `a` at it. Assigning `b = a` doesn't copy — it just adds a second label to the same object. The `id()` function reveals an object's identity:

```python
a = 5
b = a
print(id(a) == id(b))   # True — same object, two names
```

11.2 Reference counting & garbage collection

Python keeps a **reference count** for every object: how many names point at it. When that count drops to zero — nothing points to it any more — the object is destroyed and its memory is freed automatically. This is Python's **garbage collector** at work.

```python
x = "hello"   # object "hello" has 1 reference
x = "world"   # nothing points to "hello" now → it's freed
```

11.3 Immutable vs mutable

Some objects can't be changed after creation (**immutable**): `int`, `float`, `str`, `bool`, `tuple`. Others can be changed in place (**mutable**): `list`, `dict`, `set`. So “changing” a number never edits it — it makes a new object:

```python
x = 5
x = x + 1   # doesn't edit 5 — creates a new object 6, x now points to it
```

🔬 Go Deeper — the small-integer cache

Python pre-creates the integers from **−5 to 256** when it starts, and reuses them everywhere. So two variables set to `100` share one object, but two set to `1000` don't:

PYTHON

```python
a = 100; b = 100
print(a is b)   # True  (cached, same object)

a = 1000; b = 1000
print(a is b)   # False (two separate objects)
```

It's a real optimization — small numbers appear constantly, so caching them saves memory.

The takeaway

You don't manage memory by hand — but knowing that names are labels on shared objects explains a whole class of “why did that change too?” surprises you'll hit with lists later.

12 Git — Managing Your Code

Git is a **version control** system: it takes snapshots of your code over time, so you can jump back to any working version and collaborate without overwriting others.

🖥️ Git
Runs on your computer. Tracks the changes.

☁️ GitHub
A website. Stores repos online to back up & share.

12.1 One-Time Setup

TERMINAL

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

12.2 The Core Workflow

`git init` Start tracking this folder → **`git add .`** Stage your changes → **`git commit`** Save a snapshot → **`git push`** Upload to GitHub

TERMINAL

```
git init
git status                          # see what changed
git add .
git commit -m "Add greeting program"
git log                             # view history
```

Remember it this way

`git add` = put items in a box. `git commit` = seal & label the box.

12.3 Push to GitHub & .gitignore

TERMINAL

```
git remote add origin <your-repo-link>
git push -u origin main
```

Some files should never be uploaded (passwords, junk). List them in `.gitignore`:

.GITIGNORE

```
# .gitignore
secret.txt
__pycache__/
*.log
```

PRACTICE 12

1. Make a folder with a `.py` file, then run `git init → add → commit`, and confirm it with `git log`.

🎯 TODAY'S ASSIGNMENT

Build a Greeting Program

Write a program called `greet.py` that:

- Asks the user for their **name** and their **age**.
- Prints a personalized message about next year's age.

EXPECTED OUTPUT

```
Hello Rahim! Next year you will be 26.
```

Bonus (optional): also ask for their birth year, and print whether `2026 - birth_year` equals the age they typed — using `==`. This practices comparison and type conversion together.

How to Submit

1. Create a GitHub repo named `python-assignment-1`
2. Add `greet.py`, then `add → commit → push`
3. Submit the link to your repository

Before you submit — avoid the two classic bugs

1) Convert age with `int()`, or `age + 1` throws a **TypeError**. 2) After pushing, open your repo in the browser and confirm `greet.py` is actually there.

🔬 **Stretch goal (for the curious)**

Make your program repeat the age question if the user types letters instead of a number, so it never crashes. Hint: you'll need `try / except` — we'll cover it soon, but see if you can figure it out.

📚 TAKE-HOME HOMEWORK

Five Problems to Submit

Do these **after class, at home**, and submit them together. Put each solution in its own `.py` file inside a GitHub repo named `python-homework-1`, then share the repo link. Everything you need is in these notes — flip back whenever you're stuck.

1 Temperature converter

Ask the user for a temperature in Celsius, then print it in Fahrenheit using the formula `F = C * 9/5 + 32`. Remember to convert the input with `float()`.

Example → input `37` prints `98.6`

2 Simple bill calculator

Ask for a product's **price** and **quantity**, then print the total. Add a 5% tax on top and print the final amount too. Use meaningful variable names in `snake_case`.

3 Odd or even checker

Ask the user for a whole number and print whether it is odd or even. Use the modulo operator: `number % 2 == 0` is `True` for even numbers.

Tip: for now, just print the `True` / `False` result of the comparison — we'll turn it into a full message with `if/else` next class.

4 Break it, then fix it

Write a program that **intentionally** causes a `TypeError` (hint: add a number to a string from `input()`). Run it, copy the error message into a comment, then fix the code so it works. This trains you to read errors — the most important debugging skill.

5 Explore memory with `id()`

Create `a = 200` and `b = 200`, then print `a is b`. Do the same with `2000`. In a comment, explain in one line why one prints `True` and the other `False` (revisit the small-integer cache in Section 11).

Submission checklist

Five `.py` files · one repo (`python-homework-1`) · each committed with a clear message · repo pushed and visible on GitHub · link submitted.

Coming up next class → making decisions with `if / elif / else`, and working with lists to store many values.